

# 项目中文说明

---

## 1. 这个项目是做什么的？

---

这个项目是一个**信息检索课程作业**，目标是做一个“混合搜索系统”。

简单来说，用户输入一个查询句子，系统要从一批文档里找出**最相关**的结果。

这个项目现在的核心思路是：

- 一条路用 **BM25** 做关键词匹配
- 一条路用 **Dense Embedding** 做语义匹配
- 最后把两条路的结果做融合，得到最终排序

所以它不是单纯的关键词搜索，也不是单纯的语义搜索，而是一个 **Hybrid Search (混合检索)** 系统。

---

## 2. 为什么要做“混合搜索”？

---

只用一种方法通常不够好。

### 2.1 BM25 的优点和缺点

BM25 是课堂里常见的传统检索方法，核心看的是：

- 查询词有没有出现在文档里
- 出现了几次
- 这个词在整个语料里是不是稀有词

它的优点：

- 对**精确术语、专有名词、缩写、编号**很敏感
- 可解释性强
- 实现清楚，适合课程作业

它的缺点：

- 如果查询和文档表达不同，但意思相近，BM25 可能找不到
- 例如同义词、改写表达、自然语言问法，BM25 可能表现一般

### 2.2 Dense Embedding 的优点和缺点

Dense Embedding 的思路是：

- 用预训练语言模型把一段文本变成一个向量
- 查询也变成一个向量
- 向量越接近，说明语义越相似

它的优点：

- 能处理**同义表达**
- 能处理“词面不一样，但意思相近”的情况
- 更接近现代语义搜索系统

它的缺点：

- 对某些必须精确匹配的词，不一定比 BM25 更稳
- 模型计算更重，第一次构建索引会更慢

## 2.3 为什么要融合？

因为这两类方法各有长处。

理论上，我们希望：

- BM25 负责抓住精确匹配
- Dense 负责抓住语义匹配
- 最后融合后比单独使用其中一个更强

这就是本项目的核心出发点。

---

## 3. 这个项目最开始是什么样的？

项目最开始是：

- BM25 + TF-IDF

也就是说，第二条检索路线原本不是 Dense，而是 TF-IDF。

后来我们发现：

- TF-IDF 本质上还是偏“词面匹配”
- 它虽然也是向量方法，但不是真正的语义表示
- 在需要语义理解的数据集上，效果不够好

所以后面我们把它换成了：

- BM25 + Dense Embedding

这也是目前项目的主要升级点。

---

## 4. 现在项目的当前版本在做什么？

当前版本已经完成了以下工作：

### 4.1 检索架构升级

项目现在不是 BM25 + TF-IDF，而是：

- BM25 + Dense Embedding + RRF 融合

这里的 **RRF** 是 **Reciprocal Rank Fusion**，可以理解成一种“把多个排序结果合并”的简单稳健方法。

它的作用是：

- BM25 会给出一个排序
- Dense 会给出另一个排序
- RRF 把两个排序合成最终结果

## 4.2 使用了真实的 dense 模型

现在项目不是 mock，也不是手写假向量。

当前 dense 路线使用的是真实模型：

- `BAAI/bge-small-en-v1.5`

这是通过 `sentence-transformers` 加载的。

也就是说：

- 正常运行时用的是真实 embedding 模型
- 只有单元测试里为了稳定和快速，才用了 fake encoder

## 4.3 做了结构重构

为了让代码更清楚，项目已经从“所有逻辑都堆在一个文件里”改成了分模块结构。

现在主要模块大致是：

- `engine.py`
  - 负责整体调度
- `retrievers/bm25.py`
  - BM25 检索器
- `retrievers/dense.py`
  - Dense 检索器
- `embedding.py`
  - 模型加载和文本编码
- `fusion.py`
  - RRF 融合
- `cache.py`
  - Dense cache 的保存和读取

这样做的好处是：

- 代码职责更清楚
- 更方便解释给老师和队友
- 后面如果要继续扩展，比如加 reranker 或 FAISS，会更容易

## 4.4 加了缓存机制

Dense 检索比 BM25 重很多，因为要把文档编码成 embedding。

如果每次运行都重新编码所有文档，会很慢。

所以现在项目加了缓存：

- 文档索引和 dense cache 会保存到 `artifacts/cache/`

例如：

- `artifacts/cache/hybridfind_index.json`

- `artifacts/cache/hybridfind_dense_cache.json`

这样做的意义是：

- 第一次建立索引会慢一些
- 之后搜索和评测就可以复用缓存
- 不用每次都重新编码文档

---

## 5. 当前项目只保留了哪个数据集？

现在项目已经**收敛到 SciFact 主线**。

也就是说：

- `Cranfield` 相关数据、代码、测试已经移除
- 当前只保留 `SciFact`

这样做是为了：

- 项目结构更干净
- 避免保留不用的历史代码
- 让 report 和演示的主线更聚焦

当前数据主要在：

- `data/scifact/scifact/`

---

## 6. SciFact 是什么？为什么用它？

SciFact 是一个常见的信息检索/事实验证相关数据集。

对这个项目来说，可以简单理解成：

- 它有文档集合
- 有查询
- 有标准答案 (qrels)
- 所以我们可以客观评测检索效果

使用这种标准数据集的好处是：

- 不只是“看起来能搜”
- 而是可以真正算指标，比较方法优劣

---

## 7. 我们现在主要看哪些评测指标？

项目里目前主要看这些指标：

- `Precision@10`
- `Recall@10`
- `MAP`

- MRR
- nDCG@10

如果队友只记一个大概含义，可以这样理解：

- Precision@10
  - 前 10 个结果里有多少是真的相关
- Recall@10
  - 该找出来的相关文档里，前 10 个找回了多少
- MAP
  - 综合看整体排序质量
- MRR
  - 第一个正确结果出现得有多靠前
- nDCG@10
  - 更关注高排名结果是否真的排得好

在 report 里，最值得重点讲的通常是：

- MAP
- MRR
- nDCG@10

---

## 8. 我们目前已经得到什么实验结论？

### 8.1 Dense 替换 TF-IDF 是有效的

我们已经在 SciFact 上做过对比，结论很明确：

- Dense 明显优于原来的 TF-IDF

也就是说，这次升级不是“换个更复杂的词”，而是确实带来了效果提升。

### 8.2 最优结果不是 0.5 / 0.5

我们还做了权重实验，测试了：

- 0.5 / 0.5
- 0.1 / 0.9
- 0.2 / 0.8
- 0.3 / 0.7

这里的意思是：

- BM25 weight / Dense weight

结果表明：

- 最好的不是平均分配
- 当前在 SciFact 上表现最好的是 0.1 / 0.9

这说明：

- Dense 是主力
- BM25 仍然有价值
- 但 BM25 更适合作为“小幅补充”，而不是和 Dense 五五开

## 8.3 这确实出现了“混合优于单路”的现象

在最佳权重下：

- 混合检索略微超过了 `dense_only`

所以可以说：

- 当前项目已经观察到了“少量 BM25 + 强 Dense”带来的协同效果

不过要注意：

- 并不是任何权重都会更好
- 如果 BM25 权重太大，反而会把结果拉差

这也是一个很适合在 report 里讲的点。

---

## 9. 现在仓库里哪些文件最值得看？

如果队友只是想快速理解项目，建议按下面顺序看：

### 9.1 先看文档

- `README.md`
  - 英文版/简版项目说明
- `docs/experiments/EXPERIMENT_NOTES_SCIFACT.md`
  - 已跑实验的结果记录

### 9.2 再看代码主线

- `src/hybridfind/__main__.py`
  - CLI 入口，能看清项目如何运行
- `src/hybridfind/engine.py`
  - 主引擎
- `src/hybridfind/retrievers/bm25.py`
  - BM25 实现
- `src/hybridfind/retrievers/dense.py`
  - Dense 实现
- `src/hybridfind/evaluation.py`
  - 评测逻辑

## 9.3 最后看实验结果

- `artifacts/reports/`

这里保存了实验跑出来的结果文件。

---

## 10. 这个项目怎么运行？

### 10.1 安装依赖

```
pip install -e .
```

### 10.2 建立索引

```
hybridfind index docs/ --extensions ".txt,.md"
```

这一步会：

- 读取文档
- 建 BM25
- 建 dense embedding
- 保存缓存到 `artifacts/cache/`

### 10.3 搜索

```
hybridfind search "query" --bm25-weight 0.1 --dense-weight 0.9
```

### 10.4 跑 SciFact 评测

```
hybridfind evaluate --data-dir data/scifact/scifact --dataset scifact --eval-k 10
```

如果要把结果保存出来：

```
hybridfind evaluate --data-dir data/scifact/scifact --dataset scifact --eval-k 10  
--output-csv artifacts/reports/scifact_run.csv --output-json  
artifacts/reports/scifact_run.json
```

---

## 11. 队友如果要写 report，可以怎么理解这个项目？

可以把 report 主线理解成下面这条故事线：

### 第一阶段：问题背景

- 只用关键词检索不够
- 只用传统 TF-IDF 也不够表达语义

## 第二阶段：方法设计

- 保留 BM25 作为传统检索基线
- 用 Dense Embedding 代替 TF-IDF
- 用 RRF 做混合融合

## 第三阶段：工程改进

- 把代码结构拆清楚
- 加入 dense cache，解决重复编码慢的问题
- 清理无关数据集，只保留 SciFact 主线

## 第四阶段：实验结论

- Dense 明显优于 TF-IDF
- 最优权重不是平均分配，而是偏 Dense
- 少量 BM25 能提供补充收益

如果按这个顺序讲，老师和听众会更容易跟上。

---

## 12. 队友如果要做展示，可以重点讲什么？

做演示时，不建议一上来就讲很多代码。

更建议按下面顺序讲：

1. 我们想解决什么问题
2. BM25 和 Dense 分别是什么
3. 为什么要做混合检索
4. 为什么要把 TF-IDF 换成 Dense
5. SciFact 上实验结果如何
6. 为什么 0.1 / 0.9 最优
7. 工程上我们怎么做了缓存和结构整理

也就是说：

- 先讲“为什么”
- 再讲“怎么做”
- 最后讲“效果如何”

---

## 13. 当前项目可以一句话怎么概括？

一句话版本可以这样说：

这是一个基于 BM25 和 Dense Embedding 的混合信息检索系统，我们用真实语义向量替换了原来的 TF-IDF，并在 SciFact 数据集上验证了效果提升，同时完成了缓存和代码结构重构，方便后续写报告和做演示。

---



## 14. 目前项目的状态总结

---

当前项目已经具备：

- 清晰的混合检索主线
- 真实 dense 模型
- 默认缓存机制
- SciFact 评测能力
- 已完成的实验结果记录

所以它现在不是一个“只有代码雏形”的项目，而是一个已经可以：

- 讲原理
- 展示结构
- 跑实验
- 写报告

的课程作业版本。